Programming 2

Tutorial 7

Activity 1 (required)

Implement a simple student management system based on the design below. The system should store personal information (name, date of birth, student ID), GPA, and a list of registered courses for each student.

Components:

1. Class: `Student`

- Fields (private):
  - `name: String` (student's full name)
  - `dateOfBirth: LocalDate` (date of birth)
  - `studentId: String` (unique student ID)
  - `gpa: double` (grade point average)
  - `grades: Map<Course, Double>` (mapping of enrolled courses to grades)
- Constructors:
  - `Student(String name, LocalDate dob, String sId)`: Initialize student object with basic information.
- Methods (public unless noted):
  - Getters for all fields.
  - `registerCourse(Course course)`: adds a course to the student's record (without grade yet).
  - `assignGrade(Course course, double grade)`: assigns a grade for a registered course.
  - `calculateGPA()`: recalculates GPA as weighted average of all assigned grades based on course credits.
  - `toString()`: returns a formatted string with student details, GPA, and enrolled courses.

2. Interface: `Course`

- Methods:
  - `getName()`: Return the name of the course (public).
  - `getCourseCode()`: returns unique course code.
  - `getCredits()`: returns number of credits.

3. Concrete Class: `JavaCourse` (implementing `Course`)

- Fields (private):
  - `name`: String (course name)
  - `courseCode: String` (unique course identifier)
  - `credits: int` (number of credits)
- Constructor:
  - `JavaCourse(String name, String courseCode, int credits)`: initializes course object.
- Methods: Implement all interface methods (`getName()`, `getCourseCode()`, `getCredits()`).

4. Anonymous Class:

Use an anonymous class to create a one-off course inline that implements `Course`.

E.g. enroll a "Special Subject" course for a student without defining a separate class.

Activity 2 (required)

Design and implement a library management system using OOP concepts in Java.

Components:

1. Interface: `Item`

- Methods (public):
  - `getId(): int` (returns the item ID)
  - `getTitle(): String` (returns the item title)
  - `getAuthor(): String` (returns the item author)
  - `getType(): ItemType` (returns the item type, use an enum instead of String)

2. Enum: `ItemType`

Defines item categories to avoid string duplication.

Values: `BOOK`, `JOURNAL`, `DVD`

3. Abstract Class: `AbstractItem` implementing `Item`

- Fields (private):
  - `id: int`
  - `title: String`
  - `author: String`
  - `type: ItemType`
- Methods:
  - Standard getters (`getId`, `getTitle`, `getAuthor`, `getType`)
  - `toString(): String` (returns formatted item information)
- Subclasses:
  - `Book` – addsfields like number of pages, publisher, publication year.
  - `Journal` – adds fields like issue number, publication date.
  - `DVD` – adds fields like duration, director, actors.

4. Class: `Person`

- Fields:
  - `id: int`
  - `name: String`
  - `email: String`
- Methods (public):
  - Getters for all fields.
  - `toString()`: returns person information.

5. Class: `Member` (subclass of `Person`)

- Fields (private):
  - `borrowedItems: List<Item>`
- Methods:
  - `getBorrowedItems(): List<Item>` (returns an unmodifiable list)
  - `borrowItem(Item item): void` (borrows an item, does nothing and prints a warning message if already borrowed)

    - `returnItem(Item item): void` (returns an item, does nothing and prints a warning message if not borrowed)
    - `toString(): String` (overrides to include borrowed items)

6. Class: `Library`

- Fields (private):
    - `items: List<Item>` (list of all items)
    - `members: List<Member>` (list of all members)
- Methods (public unless noted otherwise):
    - `addItem(Item item)`: Add an item to the library.
    - `removeItem(Item item)`: Remove an item from the library.
    - `findItemById(int id)`: Find an item by ID.
    - `findItemByTitle(String title)`: Find an item by title.
    - `addMember(Member member)`: Add a member to list of members
    - `removeMember(int memberId)`: Remove a member from the list of members
    - `findMemberById(int id)`: Find and return a member by ID
    - `borrowItem(int memberId, int itemId)`: Allow a member to borrow an item.
    - `returnItem(int memberId, int itemId)`: Allow a member to return an item.
    - `printAllItems()`: Print information of all items.
    - `printAllMembers()`: Print information of all members.

7. `Main` Class:

Demonstrates usage:

- Initialize library.
- Add items and members.
- Perform borrow/return operations.
- Print all items and members.

Submission

Submit a **zip** file containing all of your source codes and documents to this tutorial's submission box in the course website on FIT LMS.